CODE OUTLINE DOCUMENT

Posit-Optimised YOLOv8n FPGA Accelerator

Verilog / SystemVerilog • Posit⟨16,1⟩ • 1D Systolic PDPU Array • ZCU104

Companion to: PRD v0.3 — Posit-Optimised YOLOv8n on FPGA
Language: Mixed: Verilog (RTL) + SystemVerilog (testbenches + packages)
Reset style: Active-high synchronous reset (rst)
Flow control: Single enable signal per pipeline stage
Clock: Single clock domain (clk); frequency decided in Vivado
Parameters: Shared via SV header file: posit_defines.svh
Naming: module_portname convention; pdpu_ prefix (arithmetic); yolo_ prefix (network)
Version: 0.1 — Draft

0. How to Use This Document

This document is the authoritative code specification for the Posit YOLOv8n FPGA accelerator. Each section corresponds to one or more HDL source files. An agent or developer assigned to implement a single module should:

- Read Section 1 (Global Conventions) before anything else — it defines naming rules, the shared header file, and reset/enable conventions that all modules must follow.
- Read the module box for their assigned module — each box contains: language, filename, all parameters with widths and defaults, all ports with directions and widths, key internal signal names, full behaviour description, dependencies (what it instantiates), and testbench requirements.
- Check the dependency list — if a module instantiates another, that module's box defines its exact interface. Do not invent port names.
- Do not change parameter names, port names, or signal names defined here without updating this document — other modules depend on these exact names.
- Implement only what is specified. Mark unresolved open items (flagged as TODO in each box) with a // TODO comment in the RTL and notify the team.

Module boxes are colour-coded: blue header = module name and file. Orange labels = section within the box. Green italics (in code blocks) = comments. All port and signal names are in Courier New monospace.

---

1. Global Conventions

1.1 File and Module Naming

Prefix	Domain	Example
pdpu_	Posit arithmetic units	pdpu_mitchell_mult, pdpu_approx_adder, pdpu_pe, pdpu_array
yolo_	Network and datapath	yolo_im2col, yolo_layer_ctrl, yolo_conv, yolo_top
tb_	Testbenches (SV)	tb_pdpu_pe, tb_pdpu_array, tb_yolo_top

1.2 Signal Naming Convention

- All signals: snake_case. e.g. weight_reg, act_in, acc_out.
- All parameters: UPPER_CASE. e.g. POSIT_WIDTH, ES, N_PES, ACC_WIDTH.
- Port names: module_name prefix dropped on the signal name; the port itself is named as the signal. e.g. port act_in on module pdpu_pe is referenced as pdpu_pe_inst.act_in at the parent.
- Registered signals: suffix _r. e.g. acc_r, weight_r.
- Next-state combinational: suffix _next. e.g. acc_next, state_next.
- Active-high enable: en. Active-high valid: valid. Active-high reset: rst.
- Bus indices: [MSB:0] throughout. Posit sign bit always at [POSIT_WIDTH-1].

1.3 Shared Header File: posit_defines.svh

Every module must include this file at the top. Do not define parameters locally if they are listed here.

```
`ifndef POSIT_DEFINES_SVH
`define POSIT_DEFINES_SVH

// ── Posit format ──────────────────────────────────────
`define POSIT_WIDTH   16
`define ES            1
`define REGIME_MAX    3    // max absolute regime value k
`define USEED         4    // 2^(2^ES)

// ── Accumulator ───────────────────────────────────────
`define ACC_WIDTH     48   // TBD: 32 or 48 — update when decided

// ── Array ─────────────────────────────────────────────
`define N_PES_MAX     16   // maximum PE count (T3 upper bound)
`define N_PES         4    // active PE count — set per tier

// ── Im2col ────────────────────────────────────────────
`define KERNEL_MAX    3    // max kernel dimension
`define MAX_CIN       512  // max input channels

// ── BRAM ──────────────────────────────────────────────
`define BRAM_ADDR_W   16
`define BRAM_DATA_W   `POSIT_WIDTH
```

```
// ── Reset / flow control ──────────────────────────────
// rst: active-high synchronous
// en:  active-high enable per stage (no backpressure)

`endif // POSIT_DEFINES_SVH
```

1.4 Reset and Enable Protocol

- rst = 1 holds all registers at their reset value for the entire cycle it is asserted. All always_ff / always blocks must check rst first.
- en = 1 allows a stage to advance. en = 0 freezes the stage (registers hold). This is the only flow control mechanism between stages.
- Template for all registered modules:

```
always_ff @(posedge clk) begin
    if (rst) begin
        signal_r <= RESET_VALUE;
    end else if (en) begin
        signal_r <= signal_next;
    end
end
```

1.5 Directory Structure

```
rtl/
  posit_defines.svh          // shared header — include everywhere
  arithmetic/
    pdpu_mitchell_mult.v      // Mitchell approximate multiplier
    pdpu_approx_adder.v       // Approximate adder (TBD variant)
    pdpu_pe.sv                // Single Processing Element
    pdpu_reduction_tree.sv    // Binary adder tree
    pdpu_array.sv             // 1D systolic array of N_PES PEs
  datapath/
    yolo_im2col.sv            // Im2col patch extraction buffer
    yolo_weight_bram.sv       // Weight BRAM bank per PE
    yolo_feat_bram.sv         // Feature map double-buffer BRAM
    yolo_layer_ctrl.sv        // Layer sequencer FSM
    yolo_posit_compare.v      // Exact Posit16 comparator (SPPF MaxPool)
    yolo_residual_add.sv      // Posit16 residual add (C2f shortcuts)
  top/
    yolo_top.sv               // Top-level: array + ctrl + memory + AXI
tb/
  tb_pdpu_mitchell_mult.sv
  tb_pdpu_approx_adder.sv
  tb_pdpu_pe.sv
  tb_pdpu_array.sv
  tb_yolo_im2col.sv
  tb_yolo_layer_ctrl.sv
  tb_yolo_top.sv
```

---

2. Arithmetic Layer — pdpu_ Modules

These modules form the compute core. Implement and verify in Tier 1 order: mitchell_mult → approx_adder → pe → reduction_tree → array. Each must pass its testbench before the next is started.

2.1 pdpu_mitchell_mult

pdpu_mitchell_mult [Verilog] rtl/arithmetic/pdpu_mitchell_mult.v

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH   // total bit width
parameter ES          = `ES            // exponent bits
parameter REGIME_MAX  = `REGIME_MAX    // max |k|
```

PORTS

```
input  wire                    clk
input  wire                    rst
input  wire                    en
input  wire [POSIT_WIDTH-1:0]  mult_a      // Posit16 operand A
input  wire [POSIT_WIDTH-1:0]  mult_b      // Posit16 operand B
output reg  [POSIT_WIDTH-1:0]  mult_result // Posit16 product
output reg                     mult_valid  // result valid next cycle
```

KEY INTERNAL SIGNALS / REGISTERS

```
wire        sign_a, sign_b, sign_r
wire [2:0]  k_a, k_b           // regime values, capped at REGIME_MAX
wire [ES-1:0] exp_a, exp_b
wire [POSIT_WIDTH-ES-4:0] frac_a, frac_b  // fraction fields
wire [POSIT_WIDTH-2:0]  log_a, log_b       // log-domain integer repr
wire [POSIT_WIDTH-1:0]  log_sum            // L_A + L_B
wire [POSIT_WIDTH-1:0]  log_corrected      // after frac correction
wire [POSIT_WIDTH-ES-4:0] frac_corr        // fA x fB correction term
reg  [POSIT_WIDTH-1:0]  mult_result_next
```

BEHAVIOUR / IMPLEMENTATION NOTES

STAGE 1 — DECODE (combinational):

Extract sign, regime k (run-length decode, cap at REGIME_MAX), exponent, fraction from mult_a and mult_b.

Regime decoder: scan bits after sign; count run of identical bits; k = run_length if first regime bit = 1, else k = -(run_length+1). Clamp: if |k| > REGIME_MAX set k = sign(k)*REGIME_MAX.

Log encoding: log_a = {k_a*(2^ES) + exp_a, frac_a} packed as fixed-point. Same for log_b.

STAGE 2 — LOG-DOMAIN ADD (combinational):

log_sum = log_a + log_b (integer addition of log-domain representations).

STAGE 3 — ERROR CORRECTION (combinational):

frac_corr = frac_a * frac_b (sub-word multiply; see Open Item #7 — LUT or shift-add).

log_corrected = log_sum - frac_corr.

TODO (Open Item #7): replace this line with chosen implementation (LUT or shift-and-add).
STAGE 4 — ANTILOG / ENCODE (combinational):
Reconstruct Posit16 from log_corrected: integer part -> regime+exponent fields; fractional part -> fraction field.
Handle saturation: if reconstructed |k| > REGIME_MAX, clamp to NaR or max representable (TODO Open Item #1).
STAGE 5 — SIGN (combinational):
sign_r = sign_a XOR sign_b. Apply to result.
SPECIAL CASES (must be handled before stages 1-5):
If mult_a == 0 or mult_b == 0: result = 0.
If mult_a == NaR or mult_b == NaR: result = NaR.
NaR encoding = {1'b1, {(POSIT_WIDTH-1){1'b0}}} (MSB=1, rest 0).
REGISTERED OUTPUT:
On posedge clk, if en: mult_result <= mult_result_next; mult_valid <= 1.
Latency: 1 clock cycle (all combinational stages registered at output).
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: nothing (leaf module)
Used by: pdpu_pe
TESTBENCH NOTES [SV]
tb_pdpu_mitchell_mult.sv must:
- Drive 10,000+ random Posit16 (A,B) pairs.
- Compute reference using SoftPosit library (C DPI or pre-generated golden vectors).
- Calculate MRE = |result - ref| / |ref| for each pair; assert MRE < 0.01 (1%) on average.
- Test all special cases: 0*x, NaR*x, x*0, NaR*NaR.
- Test regime saturation: inputs with |k| = REGIME_MAX, REGIME_MAX+1.
- Log per-test MRE to a CSV file for error distribution plotting.

2.2 pdpu_approx_adder

pdpu_approx_adder [Verilog] rtl/arithmetic/pdpu_approx_adder.v
PARAMETERS
`parameter POSIT_WIDTH = `POSIT_WIDTH`
`parameter ES = `ES`
`parameter REGIME_MAX = `REGIME_MAX`
PORTS

```
input  wire                    clk
input  wire                    rst
input  wire                    en
input  wire [POSIT_WIDTH-1:0]  add_a
input  wire [POSIT_WIDTH-1:0]  add_b
output reg  [POSIT_WIDTH-1:0]  add_result
output reg                     add_valid
```

KEY INTERNAL SIGNALS / REGISTERS

```
// Internal signals depend on chosen approximate adder variant.
// TODO (Open Item #2): fill in once variant is confirmed.
// Placeholder signals for structural shell:
wire [POSIT_WIDTH-2:0]  sig_a, sig_b     // aligned significands
wire [2:0]              k_a, k_b          // decoded regime values
wire [ES-1:0]           exp_a, exp_b
wire signed [3:0]       k_diff            // regime alignment shift
reg  [POSIT_WIDTH-1:0]  add_result_next
```

BEHAVIOUR / IMPLEMENTATION NOTES

TODO: This module is a STRUCTURAL SHELL until Open Item #2 is resolved.
Implement as an exact Posit16 adder first (from Posit-HDL-Arithmetic),
then swap to the approximate variant. This lets the rest of the
pipeline proceed while the adder is being decided.

EXACT ADDER (temporary, for pipeline unblocking):

STEP 1 — DECODE both operands: sign, k, exp, frac.
STEP 2 — ALIGN: compute scale difference between operands.
Shift smaller operand significand right by |scale_diff|.
STEP 3 — ADD/SUBTRACT aligned significands (sign-magnitude).
STEP 4 — NORMALISE result: leading-zero detect; adjust regime/exp.
STEP 5 — ENCODE back to Posit16; apply saturation policy.
STEP 6 — Register output; assert add_valid.

APPROXIMATE VARIANT (fill in when Open Item #2 resolved):

Document variant name, reference, and error characterisation here.
Replace steps 3-4 with the approximate adder logic.
Keep decode (step 1-2) and encode (steps 5-6) identical.

SPECIAL CASES: a==0 -> result=b; b==0 -> result=a; NaR input -> result=NaR.

DEPENDENCIES (instantiates / imports)

Includes: posit_defines.svh
Instantiates: nothing (leaf module)
Used by: pdpu_pe, pdpu_reduction_tree, yolo_residual_add

TESTBENCH NOTES [SV]
tb_pdpu_approx_adder.sv must:
- Drive 10,000+ random Posit16 (A,B) pairs.
- Compare to SoftPosit exact reference; measure MRE and error distance.
- Test special cases: 0+x, NaR+x, x+(-x) (cancellation).
- After approximate variant is integrated: re-run and compare exact vs approx MRE.
- Log results to CSV for error histogram.

2.3 pdpu_pe (Processing Element)

pdpu_pe [SystemVerilog] rtl/arithmetic/pdpu_pe.sv

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
parameter ES          = `ES
parameter ACC_WIDTH   = `ACC_WIDTH    // wide integer accumulator width
parameter REGIME_MAX  = `REGIME_MAX
```

PORTS

```
input  wire                   clk
input  wire                   rst
input  wire                   en
input  wire                   pe_load_weight  // 1 = load weight_in to reg; 0 = compute
input  wire                   pe_acc_clear    // 1 = clear accumulator this cycle
input  wire [POSIT_WIDTH-1:0] pe_weight_in    // weight from BRAM
input  wire [POSIT_WIDTH-1:0] pe_act_in       // activation from left neighbour (or
input)
output reg  [POSIT_WIDTH-1:0] pe_act_out      // activation forwarded to right neighbour
output reg  [ACC_WIDTH-1:0]   pe_acc_out      // accumulated partial sum (wide integer)
output reg                    pe_acc_valid     // high when acc holds a complete dot
product
```

KEY INTERNAL SIGNALS / REGISTERS

```
reg  [POSIT_WIDTH-1:0] weight_r        // stationary weight register
reg  [ACC_WIDTH-1:0]   acc_r           // wide integer accumulator
wire [POSIT_WIDTH-1:0] mult_result     // output of Mitchell multiplier
wire                   mult_valid
wire [ACC_WIDTH-1:0]   mult_extended   // mult_result sign-extended to ACC_WIDTH
reg  [POSIT_WIDTH-1:0] act_pipe_r      // 1-hop pipeline register for act passthrough
```

BEHAVIOUR / IMPLEMENTATION NOTES

WEIGHT LOAD PHASE (pe_load_weight == 1):

On posedge clk, if en & pe_load_weight: weight_r <= pe_weight_in.

acc_r is NOT cleared here (separate pe_acc_clear signal).

pe_act_out forwards pe_act_in unchanged (1-hop register).

ACCUMULATOR CLEAR:
On posedge clk, if en & pe_acc_clear: acc_r <= 0.
This happens at the start of each new dot product.
pe_acc_clear and pe_load_weight can be asserted together if a new weight
is loaded simultaneously with starting a new dot product.
COMPUTE PHASE (pe_load_weight == 0, pe_acc_clear == 0):
Mitchell multiplier is always running combinationally on weight_r and pe_act_in.
On posedge clk, if en & mult_valid:
acc_r <= acc_r + mult_extended.
(mult_extended = sign-extended Posit16 result interpreted as a signed integer
for accumulation — NOTE: this interpretation TBD; discuss with team.)
pe_act_out <= pe_act_in (1-hop shift register, always active when en=1).
ACC OUTPUT:
pe_acc_out <= acc_r (registered).
pe_acc_valid: assert high for one cycle when the layer controller signals
dot product completion (done signal from yolo_layer_ctrl, connected externally).
RESET: weight_r <= 0; acc_r <= 0; act_pipe_r <= 0; pe_acc_valid <= 0.
LATENCY: Mitchell mult = 1 cycle; accumulation starts cycle after weight loaded.
NOTE: 1-hop means act_out is delayed by 1 cycle relative to act_in.
PE[k] receives the activation that PE[k-1] received one cycle earlier.
The layer controller must account for the N_PES-1 cycle fill latency.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: pdpu_mitchell_mult (one instance)
Used by: pdpu_array
TESTBENCH NOTES [SV]
tb_pdpu_pe.sv must:
- Load a known weight; drive a sequence of activation values.
- After dot_product_len cycles, compare acc_r to expected value computed in SV.
- Test pe_acc_clear: clear mid-accumulation; verify acc resets to 0.
- Test pe_load_weight: change weight; verify new weight used immediately.
- Probe pe_act_out to verify 1-hop delay.
- Drive en=0 mid-computation; verify acc holds value (no change).

2.4 pdpu_reduction_tree

pdpu_reduction_tree [SystemVerilog] rtl/arithmetic/pdpu_reduction_tree.sv

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
parameter ACC_WIDTH   = `ACC_WIDTH
parameter N_PES       = `N_PES      // must be power of 2: 4, 8, or 16
```

PORTS

```
input  wire                              clk
input  wire                              rst
input  wire                              en
input  wire                              tree_start      // pulse: begin reduction
input  wire [N_PES-1:0][ACC_WIDTH-1:0]  tree_acc_in     // packed PE accumulator array
output reg  [POSIT_WIDTH-1:0]            tree_result     // final Posit16 output
output reg                               tree_result_valid
```

KEY INTERNAL SIGNALS / REGISTERS

```
// Intermediate sums at each tree level
// Level 0 = PE outputs (ACC_WIDTH wide)
// Levels 1..log2(N_PES) = summed and narrowed
logic [ACC_WIDTH-1:0] level [0:$clog2(N_PES)][0:N_PES-1]
logic [POSIT_WIDTH-1:0] rounded_result  // result after Posit16 rounding
logic                   round_valid
```

BEHAVIOUR / IMPLEMENTATION NOTES

TREE STRUCTURE (generate block recommended):

Level 0: level[0][i] = tree_acc_in[i] for i in 0..N_PES-1.

Level l (l=1..log2(N_PES)):

level[l][i] = level[l-1][2i] + level[l-1][2i+1] (integer add, ACC_WIDTH wide).

Final sum: level[log2(N_PES)][0] is the wide integer dot product result.

ROUNDING (combinational after tree):

Convert wide integer sum to Posit16 round-to-nearest.

This is the ONLY rounding operation in the entire dot product.

Rounding logic: extract sign from MSB of sum; scale to Posit16 range;

encode regime, exponent, fraction. Clamp if |k| > REGIME_MAX.

TODO: implement as a separate sub-function or small module if complex.

REGISTERED OUTPUT:

On posedge clk, if tree_start & en:

tree_result <= rounded_result.

tree_result_valid <= 1.

Next cycle: tree_result_valid <= 0 (pulse).

NOTE ON ADDER TYPE IN TREE:

Tree levels use INTEGER addition (not Posit addition) on wide accumulators.
The approximate Posit adder (pdpu_approx_adder) is NOT used here.
Only the final rounding step produces a Posit16 value.
This avoids error accumulation across tree levels.
RESET: all level registers = 0; tree_result = 0; tree_result_valid = 0.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: nothing (uses generate for tree levels)
Used by: pdpu_array
TESTBENCH NOTES [SV]
tb_pdpu_reduction_tree.sv:
- Drive N_PES known ACC_WIDTH integer values.
- Assert tree_start; check tree_result matches expected sum rounded to Posit16.
- Test all supported N_PES values: 4, 8, 16.
- Test tree with all-zero inputs; verify result = Posit16 zero.
- Test with one large value and N_PES-1 zeros; verify correct rounding.

2.5 pdpu_array (1D Systolic Array)

pdpu_array [SystemVerilog] rtl/arithmetic/pdpu_array.sv

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
parameter ES          = `ES
parameter ACC_WIDTH   = `ACC_WIDTH
parameter N_PES       = `N_PES
parameter REGIME_MAX  = `REGIME_MAX
```

PORTS

```
input  wire                    clk
input  wire                    rst
input  wire                    en
input  wire                    arr_load_weights  // 1 = weight load phase
input  wire                    arr_acc_clear     // 1 = clear all PE accumulators
input  wire                    arr_dot_done      // pulse from layer_ctrl: dot product
complete
input  wire [N_PES-1:0][POSIT_WIDTH-1:0] arr_weights  // weight vector for all PEs
input  wire [POSIT_WIDTH-1:0]  arr_act_in        // one activation element per cycle
output wire [POSIT_WIDTH-1:0]  arr_result        // Posit16 dot product output
output wire                    arr_result_valid   // one-cycle pulse per dot product
```

KEY INTERNAL SIGNALS / REGISTERS

```
wire [N_PES:0][POSIT_WIDTH-1:0]  act_chain  // activation shift chain
                                            // act_chain[0] = arr_act_in
```

```
                                                // act_chain[k] = PE[k-1] act_out
  wire [N_PES-1:0][ACC_WIDTH-1:0]  acc_outs    // partial sums from all PEs
  wire [N_PES-1:0]                 acc_valids // valid flags from all PEs
```

BEHAVIOUR / IMPLEMENTATION NOTES

```
GENERATE BLOCK — instantiate N_PES pdpu_pe instances:
  genvar i;
  generate for (i=0; i<N_PES; i++) begin : gen_pe
    pdpu_pe #(.POSIT_WIDTH(POSIT_WIDTH), .ES(ES),
              .ACC_WIDTH(ACC_WIDTH), .REGIME_MAX(REGIME_MAX))
    pe_inst (
      .clk          (clk),
      .rst          (rst),
      .en           (en),
      .pe_load_weight(arr_load_weights),
      .pe_acc_clear  (arr_acc_clear),
      .pe_weight_in  (arr_weights[i]),
      .pe_act_in     (act_chain[i]),
      .pe_act_out    (act_chain[i+1]),   // 1-hop shift
      .pe_acc_out    (acc_outs[i]),
      .pe_acc_valid  (acc_valids[i])
    );
  end endgenerate

ACTIVATION CHAIN:
  act_chain[0] = arr_act_in (direct connection).
  Each PE forwards its act_in to act_out via 1-hop register.
  act_chain[N_PES] is unused (rightmost PE output).

REDUCTION TREE:
  Instantiate pdpu_reduction_tree with N_PES PE accumulator outputs.
  Connect tree_start = arr_dot_done.
  arr_result      = tree_result.
  arr_result_valid = tree_result_valid.

WEIGHT LOADING:
  When arr_load_weights=1: each PE[i] loads arr_weights[i] into its weight_r.
  arr_weights is a flat bus from the weight BRAM controller.
  Weight load completes in 1 cycle (all PEs load in parallel).
```

FILL LATENCY:
First valid partial product at PE[N_PES-1] arrives N_PES-1 cycles after
the first activation is presented at arr_act_in.
Layer controller must flush N_PES-1 dummy zeros before asserting arr_dot_done.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: pdpu_pe (N_PES instances), pdpu_reduction_tree (1 instance)
Used by: yolo_top
TESTBENCH NOTES [SV]
tb_pdpu_array.sv:
- T2 test: N_PES=4; drive known 4-element weight and activation vectors.
- Load weights; assert arr_acc_clear; stream activations one per cycle.
- After dot_product_len + N_PES-1 (fill) cycles, assert arr_dot_done.
- Verify arr_result matches expected dot product computed in SV.
- Test back-to-back dot products: verify acc_clear and weight reload.
- Test with N_PES=8 and N_PES=16 (T3): change `N_PES define; re-run.
- Probe act_chain at each PE boundary to verify 1-hop stagger.

---

3. Datapath Layer — yolo_ Modules

These modules connect the array to the YOLOv8n network structure. Implement after pdpu_array passes its testbench (end of Tier 2).

3.1 yolo_im2col

yolo_im2col [SystemVerilog] rtl/datapath/yolo_im2col.sv			
PARAMETERS			
parameter POSIT_WIDTH = `POSIT_WIDTH			
parameter MAX_CIN = `MAX_CIN			
parameter KERNEL_MAX = `KERNEL_MAX			
parameter BRAM_ADDR_W = `BRAM_ADDR_W			
PORTS			
input	wire	clk	
input	wire	rst	
input	wire	en	
// Configuration (set by layer_ctrl before each layer)			
input	wire [1:0]	im2col_kernel_size	// 1 = 1x1, 3 = 3x3
input	wire [9:0]	im2col_feat_h	// input feature map height
input	wire [9:0]	im2col_feat_w	// input feature map width
input	wire [9:0]	im2col_c_in	// input channels

```
// Feature map BRAM read interface
output reg  [BRAM_ADDR_W-1:0]  im2col_bram_addr
input  wire [POSIT_WIDTH-1:0]  im2col_bram_rdata
// Output to array
output reg  [POSIT_WIDTH-1:0]  im2col_act_out      // one Posit16 element per cycle
output reg                     im2col_act_valid
output reg                     im2col_patch_done   // pulse: all elements of one patch
sent
output reg                     im2col_frame_done   // pulse: all output pixels processed
```

KEY INTERNAL SIGNALS / REGISTERS

```
reg  [9:0]  out_row_r, out_col_r    // current output pixel coordinates
reg  [3:0]  krow_r, kcol_r          // kernel scan position (0..kernel_size-1)
reg  [9:0]  cin_r                   // current input channel counter
reg  [2:0]  state_r                 // FSM state
wire [9:0]  in_row, in_col          // input pixel coords for current kernel pos
wire        is_pad                  // 1 if in_row/col out of bounds (pad with 0)
wire [BRAM_ADDR_W-1:0] feat_addr    // computed BRAM address
```

BEHAVIOUR / IMPLEMENTATION NOTES

FSM STATES:

S_IDLE: wait for en from layer_ctrl.

S_CONFIGURE: latch configuration registers; precompute address offsets.

S_SCAN: main scanning loop.

S_DONE: assert im2col_frame_done for one cycle; return to S_IDLE.

S_SCAN OPERATION (innermost loops, unrolled as FSM or counter chain):

```
for out_row in 0..feat_h-1:
  for out_col in 0..feat_w-1:
    for krow in 0..kernel_size-1:
      for kcol in 0..kernel_size-1:
        for cin in 0..c_in-1:
          in_row = out_row + krow - pad
          in_col = out_col + kcol - pad
          if (in_row < 0 or in_row >= feat_h or in_col < 0 or in_col >= feat_w):
            im2col_act_out <= Posit16_ZERO  // zero padding
          else:
            feat_addr = in_row * feat_w * c_in + in_col * c_in + cin
            im2col_bram_addr <= feat_addr  // read BRAM next cycle
            im2col_act_out   <= im2col_bram_rdata  // one cycle read latency
          im2col_act_valid <= 1
        (end cin loop) -> assert im2col_patch_done for one cycle
```

FOR 1x1 CONVOLUTION: kernel_size=1, pad=0.
Inner loops collapse: only cin loop remains.
patch length = c_in (no krow/kcol iterations).
BRAM READ LATENCY: assume 1-cycle synchronous BRAM read.
Address presented cycle N; data valid cycle N+1.
FSM must account for this: insert one wait state after address.
PAD VALUE: Posit16 zero = 16'h0000.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: nothing
Used by: yolo_layer_ctrl (receives im2col_patch_done, im2col_frame_done)
Reads from: yolo_feat_bram (input feature map bank)
TESTBENCH NOTES [SV]
tb_yolo_im2col.sv:
- Load a small synthetic 4x4x3 feature map into a BRAM model.
- Configure for 3x3 kernel; run; collect all im2col_act_out values.
- Compare against Python-generated expected im2col matrix (pre-computed golden).
- Repeat for 1x1 kernel.
- Verify padding: check boundary rows/cols produce Posit16 zero outputs.
- Verify im2col_patch_done pulses after exactly kernel*kernel*c_in elements.

3.2 yolo_weight_bram

yolo_weight_bram [SystemVerilog] rtl/datapath/yolo_weight_bram.sv

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
parameter N_PES       = `N_PES
parameter BRAM_ADDR_W = `BRAM_ADDR_W
parameter BRAM_DEPTH  = 1 << BRAM_ADDR_W
```

PORTS

```
input  wire                    clk
input  wire                    rst
input  wire                    en
// AXI write interface (from PS DMA, simplified)
input  wire                    wbram_wr_en
input  wire [BRAM_ADDR_W-1:0]  wbram_wr_addr
input  wire [POSIT_WIDTH-1:0]  wbram_wr_data
// Read interface (one bank per PE)
input  wire [BRAM_ADDR_W-1:0]  wbram_rd_addr   // same address broadcast to all banks
```

```
output reg  [N_PES-1:0][POSIT_WIDTH-1:0] wbram_rd_data  // one weight per PE
```

KEY INTERNAL SIGNALS / REGISTERS

```
// N_PES independent BRAM banks, each holding weights for one PE position
// Bank[i] holds weight[i] for output_channel[0], weight[i] for output_channel[1], ...
reg [POSIT_WIDTH-1:0] bram [0:N_PES-1][0:BRAM_DEPTH-1]
```

BEHAVIOUR / IMPLEMENTATION NOTES

WRITE PORT (from AXI DMA, PS side):

wbram_wr_addr encodes: {bank_select[log2(N_PES)-1:0], channel_offset[BRAM_ADDR_W-log2(N_PES)-1:0]}

On wr_en: bram[bank_select][channel_offset] <= wbram_wr_data.

READ PORT (to array weight loader):

wbram_rd_addr = current output channel index.

On posedge clk: wbram_rd_data[i] <= bram[i][wbram_rd_addr] for all i.

Read latency: 1 cycle (synchronous BRAM read).

All N_PES weights for the current output channel read in parallel.

IMPLEMENTATION NOTE: In Vivado, infer this as Block RAM (RAMB36E2).

Use (* ram_style = "block" *) attribute on bram array.

N_PES separate BRAM instances will be inferred.

RESET: BRAM contents do NOT reset (persistent until next DMA write).

Only control registers reset.

DEPENDENCIES (instantiates / imports)

Includes: posit_defines.svh

Instantiates: inferred BRAM (Vivado)

Used by: yolo_layer_ctrl (drives wbram_rd_addr); pdpu_array (receives wbram_rd_data)

TESTBENCH NOTES [SV]

No dedicated testbench required — exercised through tb_pdpu_array and tb_yolo_layer_ctrl.

Verify BRAM inference in Vivado synthesis report (should show RAMB36E2 primitives).

3.3 yolo_feat_bram

yolo_feat_bram [SystemVerilog] rtl/datapath/yolo_feat_bram.sv

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
parameter BRAM_ADDR_W = `BRAM_ADDR_W
parameter BRAM_DEPTH  = 1 << BRAM_ADDR_W
parameter N_BANKS     = 2   // ping-pong double buffer (bank 0 = input, bank 1 = output)
```

PORTS

```
input  wire                    clk
```

```
input  wire                    rst
input  wire                    en
input  wire                    fbram_bank_sel    // 0 = bank A active as input, 1 = bank B
// Read port (to im2col buffer)
input  wire [BRAM_ADDR_W-1:0]  fbram_rd_addr
output reg  [POSIT_WIDTH-1:0]  fbram_rd_data
// Write port (from reduction tree output)
input  wire                    fbram_wr_en
input  wire [BRAM_ADDR_W-1:0]  fbram_wr_addr
input  wire [POSIT_WIDTH-1:0]  fbram_wr_data
// AXI interface (input image load from PS, output readback to PS)
input  wire                    fbram_axi_wr_en
input  wire [BRAM_ADDR_W-1:0]  fbram_axi_wr_addr
input  wire [POSIT_WIDTH-1:0]  fbram_axi_wr_data
output reg  [POSIT_WIDTH-1:0]  fbram_axi_rd_data
input  wire [BRAM_ADDR_W-1:0]  fbram_axi_rd_addr
```

KEY INTERNAL SIGNALS / REGISTERS

```
reg [POSIT_WIDTH-1:0] bank [0:N_BANKS-1][0:BRAM_DEPTH-1]
wire active_in_bank  = fbram_bank_sel ? 1 : 0   // bank holding current layer input
wire active_out_bank = fbram_bank_sel ? 0 : 1   // bank receiving current layer output
```

BEHAVIOUR / IMPLEMENTATION NOTES

DOUBLE-BUFFER SCHEME:

Two BRAM banks: A (index 0) and B (index 1).

fbram_bank_sel toggles each layer: even layers read A, write B; odd layers read B, write A.

im2col reads from active_in_bank.

Reduction tree writes to active_out_bank.

Swap by toggling fbram_bank_sel in layer_ctrl at layer boundaries.

READ PORT (to im2col):

fbram_rd_data <= bank[active_in_bank][fbram_rd_addr] (1-cycle latency).

WRITE PORT (from reduction tree):

On fbram_wr_en: bank[active_out_bank][fbram_wr_addr] <= fbram_wr_data.

AXI PORT (PS image load and result readback):

On fbram_axi_wr_en: bank[active_in_bank][fbram_axi_wr_addr] <= fbram_axi_wr_data.

fbram_axi_rd_data <= bank[active_out_bank][fbram_axi_rd_addr] (for result readback).

BRAM INFERENCE: use (* ram_style = "block" *) attribute.

If feature maps exceed single BRAM capacity: cascade multiple BRAM instances.

URAM (288Kb) preferred for large layers (P3 at 80x80xC has large footprint).

DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: inferred BRAM/URAM
Used by: yolo_im2col (reads), pdpu_array via layer_ctrl (writes result), yolo_top (AXI port)
TESTBENCH NOTES [SV]
Verify double-buffer swap in tb_yolo_layer_ctrl.
Check BRAM/URAM inference in Vivado synthesis report.

3.4 yolo_layer_ctrl (Layer Sequencer FSM)

yolo_layer_ctrl [SystemVerilog] rtl/datapath/yolo_layer_ctrl.sv

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
parameter N_PES       = `N_PES
parameter BRAM_ADDR_W = `BRAM_ADDR_W
parameter MAX_CIN     = `MAX_CIN
```

PORTS

```
input  wire        clk
input  wire        rst
input  wire        en
// Configuration from PS (AXI-Lite registers)
input  wire [3:0]  ctrl_layer_type   // encoding: 0=Conv3x3, 1=Conv1x1, 2=C2f,
                                     // 3=SPPF, 4=Upsample, 5=Concat, 6=ResAdd
input  wire [9:0]  ctrl_feat_h, ctrl_feat_w, ctrl_c_in, ctrl_c_out
input  wire        ctrl_layer_start  // pulse from PS to begin layer
// To im2col
output reg         ctrl_im2col_en
output reg [1:0]   ctrl_kernel_size
// From im2col
input  wire        ctrl_patch_done
input  wire        ctrl_frame_done
// To array
output reg         ctrl_arr_load_weights
output reg         ctrl_arr_acc_clear
output reg         ctrl_arr_dot_done
// To weight BRAM
output reg [BRAM_ADDR_W-1:0] ctrl_wbram_rd_addr
// To feature BRAM
output reg         ctrl_fbram_bank_sel
output reg         ctrl_fbram_wr_en
output reg [BRAM_ADDR_W-1:0] ctrl_fbram_wr_addr
// From array (result)
input  wire [POSIT_WIDTH-1:0] ctrl_arr_result
```

```
  input  wire                    ctrl_arr_result_valid
  // To PS
  output reg          ctrl_layer_done   // pulse when layer complete
```

KEY INTERNAL SIGNALS / REGISTERS

```
  typedef enum logic [3:0] {
    S_IDLE, S_LOAD_WEIGHTS, S_CLEAR_ACC,
    S_COMPUTE, S_WAIT_RESULT, S_WRITE_OUTPUT,
    S_NEXT_CHANNEL, S_LAYER_DONE
  } state_t;
  state_t state_r, state_next;
  reg [9:0]  out_ch_r      // current output channel counter
  reg [9:0]  wr_addr_r     // output BRAM write address counter
  reg [9:0]  fill_cnt_r    // counts N_PES-1 fill cycles
```

BEHAVIOUR / IMPLEMENTATION NOTES

FSM MAIN SEQUENCE (Conv3x3 and Conv1x1 — the primary path):

S_IDLE:
  Wait for ctrl_layer_start pulse.
  Latch ctrl_feat_h, ctrl_feat_w, ctrl_c_in, ctrl_c_out.
  -> S_LOAD_WEIGHTS

S_LOAD_WEIGHTS:
  Assert ctrl_arr_load_weights=1.
  Drive ctrl_wbram_rd_addr = out_ch_r (current output channel).
  Wait 1 cycle (BRAM read latency + PE weight register latch).
  -> S_CLEAR_ACC

S_CLEAR_ACC:
  Assert ctrl_arr_acc_clear=1 for 1 cycle.
  -> S_COMPUTE

S_COMPUTE:
  Assert ctrl_im2col_en=1.
  Im2col streams activations to array one per cycle.
  Wait for ctrl_patch_done pulse from im2col.
  Account for N_PES-1 fill cycles: after last act element,
  hold en=1 for N_PES-1 more cycles (fill_cnt_r counts down).
  When fill_cnt_r == 0: -> S_WAIT_RESULT

S_WAIT_RESULT:

Assert ctrl_arr_dot_done=1 for 1 cycle (triggers reduction tree).
Wait for ctrl_arr_result_valid pulse.
-> S_WRITE_OUTPUT
S_WRITE_OUTPUT:
Assert ctrl_fbram_wr_en=1.
ctrl_fbram_wr_addr = wr_addr_r; increment wr_addr_r.
(output pixel written to active output BRAM bank)
-> S_NEXT_CHANNEL
S_NEXT_CHANNEL:
Increment out_ch_r.
If out_ch_r < ctrl_c_out: -> S_LOAD_WEIGHTS (next output channel).
If ctrl_frame_done from im2col AND out_ch_r == ctrl_c_out: -> S_LAYER_DONE.
Else: -> S_LOAD_WEIGHTS (next output pixel batch).
S_LAYER_DONE:
Toggle ctrl_fbram_bank_sel (ping-pong swap).
Assert ctrl_layer_done=1 for 1 cycle.
-> S_IDLE
NON-CONV LAYER TYPES:
ctrl_layer_type=3 (SPPF): bypass array; instantiate yolo_posit_compare.
FSM drives the comparator directly; result written to output BRAM.
ctrl_layer_type=4 (Upsample): address remapping only; copy BRAM src to dst.
No array, no arithmetic.
ctrl_layer_type=5 (Concat): concatenate two BRAM banks into output bank.
No arithmetic.
ctrl_layer_type=6 (ResAdd): instantiate yolo_residual_add.
FSM streams two feature map values per cycle to residual adder.
ctrl_layer_type=2 (C2f): treated as sequence of Conv layers + ResAdd.
FSM sequences through conv sub-layers then residual add.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: (drives) pdpu_array, yolo_im2col, yolo_weight_bram, yolo_feat_bram
Instantiates: yolo_posit_compare (for SPPF), yolo_residual_add (for C2f/ResAdd)
Used by: yolo_top
TESTBENCH NOTES [SV]
tb_yolo_layer_ctrl.sv:

- Model all submodules as behavioural SV tasks.
- Drive a full Conv3x3 layer with small dimensions (4x4x4 in, 4 out).
- Verify: weight loads, acc clears, im2col sequencing, result writes, bank swap.
- Drive a Conv1x1 layer; verify different patch length.
- Drive a ResAdd layer; verify yolo_residual_add instantiation path.
- Drive a SPPF layer; verify yolo_posit_compare path.

3.5 yolo_posit_compare (SPPF MaxPool)

yolo_posit_compare [Verilog] rtl/datapath/yolo_posit_compare.v

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
```

PORTS

```
input  wire [POSIT_WIDTH-1:0]  cmp_a
input  wire [POSIT_WIDTH-1:0]  cmp_b
output wire [POSIT_WIDTH-1:0]  cmp_max   // combinational output: max(a,b)
```

KEY INTERNAL SIGNALS / REGISTERS

```
wire sign_a = cmp_a[POSIT_WIDTH-1]
wire sign_b = cmp_b[POSIT_WIDTH-1]
wire [POSIT_WIDTH-2:0] mag_a = cmp_a[POSIT_WIDTH-2:0]
wire [POSIT_WIDTH-2:0] mag_b = cmp_b[POSIT_WIDTH-2:0]
```

BEHAVIOUR / IMPLEMENTATION NOTES

POSIT COMPARISON PROPERTY (key insight):

Posit numbers are ordered such that for positive values,
the unsigned integer interpretation of the bit pattern
is monotonically increasing with the numeric value.
For two positive Posit16 values: a > b iff a[14:0] > b[14:0] (unsigned).
For two negative values: a > b iff a[14:0] < b[14:0] (unsigned magnitude).
NaR (MSB=1, rest=0) is defined as +/-infinity; treat as largest in magnitude.

COMBINATIONAL MAX:

```
if sign_a == 0 and sign_b == 0:  // both positive
  cmp_max = (mag_a >= mag_b) ? cmp_a : cmp_b
elif sign_a == 1 and sign_b == 1: // both negative
  cmp_max = (mag_a <= mag_b) ? cmp_a : cmp_b
elif sign_a == 0 and sign_b == 1: // a positive, b negative
  cmp_max = cmp_a
else:                             // a negative, b positive
  cmp_max = cmp_b
```

REGISTERED VERSION: layer_ctrl can optionally register cmp_max externally.
This module is purely combinational — no clk/rst/en ports needed.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: nothing (leaf, combinational)
Used by: yolo_layer_ctrl (SPPF MaxPool path)
TESTBENCH NOTES [SV]
tb: verify max(a,b) on exhaustive set of Posit16 sign combinations.
Verify NaR handling. Verify symmetry: max(a,b)==max(b,a).

3.6 yolo_residual_add

yolo_residual_add [SystemVerilog] rtl/datapath/yolo_residual_add.sv
PARAMETERS
`parameter POSIT_WIDTH = `POSIT_WIDTH`
`parameter ES = `ES`
PORTS
`input wire clk`
`input wire rst`
`input wire en`
`input wire [POSIT_WIDTH-1:0] radd_a`
`input wire [POSIT_WIDTH-1:0] radd_b`
`output wire [POSIT_WIDTH-1:0] radd_result`
`output wire radd_valid`
BEHAVIOUR / IMPLEMENTATION NOTES
Thin wrapper around pdpu_approx_adder.
radd_a and radd_b are raw Posit16 values read directly from BRAM.
No decode/encode at this boundary — pdpu_approx_adder handles it internally.
radd_result connects directly to yolo_feat_bram write port.
radd_valid = pdpu_approx_adder's add_valid output.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: pdpu_approx_adder (one instance)
Used by: yolo_layer_ctrl (C2f residual and ResAdd layer types)
TESTBENCH NOTES [SV]
No dedicated testbench — covered by tb_pdpu_approx_adder and tb_yolo_layer_ctrl.

---

4. Top Level — yolo_top

yolo_top [SystemVerilog] rtl/top/yolo_top.sv

PARAMETERS

```
parameter POSIT_WIDTH = `POSIT_WIDTH
parameter N_PES       = `N_PES
parameter ACC_WIDTH   = `ACC_WIDTH
parameter BRAM_ADDR_W = `BRAM_ADDR_W
```

PORTS

```
input  wire         clk
input  wire         rst
// AXI-Lite control interface (from PS)
input  wire [31:0]  top_axil_wdata
input  wire [11:0]  top_axil_waddr
input  wire         top_axil_wr_en
output reg  [31:0]  top_axil_rdata
input  wire [11:0]  top_axil_raddr
// AXI4 data interface (DMA — weight and feature map load/readback)
// (simplified: wr_en, wr_addr, wr_data, rd_addr, rd_data)
input  wire         top_dma_wr_en
input  wire [31:0]  top_dma_wr_addr
input  wire [POSIT_WIDTH-1:0] top_dma_wr_data
output reg  [POSIT_WIDTH-1:0] top_dma_rd_data
input  wire [31:0]  top_dma_rd_addr
// Status to PS
output reg          top_layer_done
output reg          top_inference_done
```

KEY INTERNAL SIGNALS / REGISTERS

```
// Internal wiring between submodules
// (instantiate all submodules; wire per Section 3 port names)
wire [N_PES-1:0][POSIT_WIDTH-1:0] weight_bus   // wbram_rd_data -> arr_weights
wire [POSIT_WIDTH-1:0]  act_bus                // im2col_act_out -> arr_act_in
wire [POSIT_WIDTH-1:0]  result_bus             // arr_result -> ctrl
// AXI-Lite decoded configuration registers
reg [9:0]   cfg_feat_h, cfg_feat_w, cfg_c_in, cfg_c_out
reg [3:0]   cfg_layer_type
reg         cfg_layer_start
```

BEHAVIOUR / IMPLEMENTATION NOTES

INSTANTIATES (all connected by name following Section 1 port conventions):

```
pdpu_array      #(...) array_inst  (...)
yolo_im2col     #(...) im2col_inst (...)
yolo_weight_bram  #(...) wbram_inst  (...)
yolo_feat_bram    #(...) fbram_inst  (...)
yolo_layer_ctrl   #(...) ctrl_inst   (...)
```

AXI-LITE REGISTER MAP (top_axil_waddr):
0x00: cfg_layer_type (4-bit layer type encoding)
0x04: cfg_feat_h (10-bit)
0x08: cfg_feat_w (10-bit)
0x0C: cfg_c_in (10-bit)
0x10: cfg_c_out (10-bit)
0x14: cfg_layer_start (write 1 to start; auto-clears)
0x18: status (read): {top_inference_done, top_layer_done}
DMA ADDRESS SPACE:
0x0000_0000 - 0x0FFF_FFFF: weight BRAM banks
0x1000_0000 - 0x1FFF_FFFF: feature map BRAM (input image load / result readback)
INFERENCE SEQUENCING:
PS writes all layer configs via AXI-Lite; DMA's weights for first layer.
PS asserts cfg_layer_start; PL executes layer; top_layer_done pulses.
PS DMA's next layer weights; asserts cfg_layer_start again.
Repeat for all YOLOv8n layers.
After final head layer, top_inference_done asserted.
PS reads back feature map (detection outputs) via DMA read port.
ENABLE DISTRIBUTION:
top-level en = 1 always (driven by cfg_layer_start and ctrl FSM).
Each submodule's en is driven by ctrl_inst outputs.
DEPENDENCIES (instantiates / imports)
Includes: posit_defines.svh
Instantiates: pdpu_array, yolo_im2col, yolo_weight_bram, yolo_feat_bram, yolo_layer_ctrl
Connected to: Vivado block design (PS, AXI Interconnect, DMA, ILA debug cores)
TESTBENCH NOTES [SV]
tb_yolo_top.sv:
- Behavioural AXI-Lite master task to write config registers.
- Behavioural DMA task to load Posit16 weights and input image.
- Run one full YOLOv8n inference on one COCO image (weights + activations pre-computed).
- Read back detection outputs via DMA read port.
- Compare bounding box outputs against float32 PyTorch reference (within tolerance).
- Use Vivado co-simulation or JTAG ILA on hardware for final validation.

---

5. Testbench Strategy Summary

Testbench File	Tier	Golden Reference Source	Key Assertion
tb_pdpu_mitchell_mult.sv	T1	SoftPosit C DPI / pre-gen CSV	MRE < 1% average
tb_pdpu_approx_adder.sv	T1	SoftPosit C DPI / pre-gen CSV	MRE < threshold (TBD with variant)
tb_pdpu_pe.sv	T1	SV reference accumulator	acc_r matches expected on all test vectors
tb_pdpu_array.sv	T2	SV reference dot product	arr_result matches; 1-hop stagger verified
tb_yolo_im2col.sv	T2	Python pre-gen im2col matrix	All patch elements match golden; padding correct
tb_yolo_layer_ctrl.sv	T2/T3	Behavioural submodule models	FSM state transitions; all layer types
tb_yolo_top.sv	T3	PyTorch float32 baseline	Detection bbox within 5% mAP tolerance

Golden vector workflow: run Python script against SoftPosit or PyTorch to generate .csv files of (input, expected_output) pairs. Load these from the SV testbench using $readmemh or a file I/O task. This decouples the reference computation from the RTL simulation.

---

6. Open Items Affecting This Code Outline

#	PRD Item	Impact on Code	Where to Update
1	Saturation policy (NaR vs clamp)	pdpu_mitchell_mult and pdpu_approx_adder SPECIAL CASES section	pdpu_mitchell_mult.v and pdpu_approx_adder.v
2	Approximate adder variant	pdpu_approx_adder.v internals (currently exact shell)	pdpu_approx_adder.v — replace exact with approx logic
3	Accumulator width (32 vs 48)	ACC_WIDTH in posit_defines.svh	posit_defines.svh — change `ACC_WIDTH
4	Regime cap validation result	REGIME_MAX in posit_defines.svh; saturation logic	posit_defines.svh if cap changes; all decoders
6	SiLU implementation (LUT vs PWL)	New module yolo_silu.v or .sv needed; layer_ctrl adds new layer type	Add yolo_silu to rtl/datapath/; update layer_ctrl type encoding
7	Correction term fA x fB (LUT vs shift-add)	pdpu_mitchell_mult.v STAGE 3 implementation	pdpu_mitchell_mult.v — replace TODO placeholder
8	Final PE count (8 or 16)	N_PES in posit_defines.svh; pdpu_reduction_tree depth	posit_defines.svh — change `N_PES after T2 resource report

Changing N_PES or ACC_WIDTH in posit_defines.svh propagates automatically to all modules — this is why the shared header file is mandatory. Never hardcode these values in individual modules.